SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**Shaik Azeem**	**Reg. No.:**	**192472307**
Section / Batch:	**CSE-AI**	**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems. • Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required. • Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis. • Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I Shaik Azeem(192472307) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shaik Azeem

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q69. Sorting Techniques – Detailed Comparison of Two Methods

Analyze the performance of Selection Sort and Bubble Sort on the dataset [48, 22, 35, 10, 27, 16]. Clearly interpret the problem and explain the working principles of both algorithms. Apply both sorting methods step-by-step and show the intermediate arrays after each pass. Count the total number of comparisons and swaps in each method. Analyze their time complexities and compare their operational differences. Interpret the results and justify which method is preferable for this dataset.

Given Array

[48, 22, 35, 10, 27, 16]

A) Selection Sort

Step 1

Find the smallest element in the entire array.

Smallest = **10**

Swap **48** and **10**

Array after Pass 1:

[10, 22, 35, 48, 27, 16]

Comparisons = **5**

Swaps = **1**

Step 2

Find the smallest element from **22, 35, 48, 27, 16**

Smallest = **16**

Swap **22** and **16**

Array after Pass 2:

[10, 16, 35, 48, 27, 22]

Comparisons = **4**

Swaps = **1**

Step 3

Find the smallest element from **35, 48, 27, 22**

Smallest = **22**

Swap **35** and **22**

Array after Pass 3:

[10, 16, 22, 48, 27, 35]

Comparisons = **3**

Swaps = **1**

Step 4

Find the smallest element from **48, 27, 35**

Smallest = **27**

Swap **48** and **27**

Array after Pass 4:

[10, 16, 22, 27, 48, 35]

Comparisons = **2**

Swaps = **1**

Step 5

Find the smallest element from **48, 35**

Smallest = **35**

Swap **48** and **35**

Array after Pass 5:

[10, 16, 22, 27, 35, 48]

Comparisons = **1**

Swaps = **1**

Final Sorted Array

[10, 16, 22, 27, 35, 48]

Total Comparisons

5 + 4 + 3 + 2 + 1 = **15**

Total Swaps

1 + 1 + 1 + 1 + 1 = **5**

B) Bubble Sort

Initial Array

[48, 22, 35, 10, 27, 16]

Pass 1

Compare **48 & 22** → Swap

[22, 48, 35, 10, 27, 16]

Compare **48 & 35** → Swap

[22, 35, 48, 10, 27, 16]

Compare **48 & 10** → Swap

[22, 35, 10, 48, 27, 16]

Compare **48 & 27** → Swap

[22, 35, 10, 27, 48, 16]

Compare **48 & 16** → Swap

[22, 35, 10, 27, 16, 48]

Comparisons = **5**

Swaps = **5**

Pass 2

Compare **22 & 35** → No Swap

[22, 35, 10, 27, 16, 48]

Compare **35 & 10** → Swap

[22, 10, 35, 27, 16, 48]

Compare **35 & 27** → Swap

[22, 10, 27, 35, 16, 48]

Compare **35 & 16** → Swap

[22, 10, 27, 16, 35, 48]

Comparisons = **4**

Swaps = **3**

Pass 3

Compare **22 & 10** → Swap

[10, 22, 27, 16, 35, 48]

Compare **22 & 27** → No Swap

[10, 22, 27, 16, 35, 48]

Compare **27 & 16** → Swap

[10, 22, 16, 27, 35, 48]

Comparisons = **3**

Swaps = **2**

Pass 4

Compare **10 & 22** → No Swap

[10, 22, 16, 27, 35, 48]

Compare **22 & 16** → Swap

[10, 16, 22, 27, 35, 48]

Comparisons = **2**

Swaps = **1**

Pass 5

Compare **10 & 16** → No Swap

[10, 16, 22, 27, 35, 48]

Comparisons = **1**

Swaps = **0**

Final Sorted Array

[10, 16, 22, 27, 35, 48]

Total Comparisons

5 + 4 + 3 + 2 + 1 = **15**

Total Swaps

5 + 3 + 2 + 1 + 0 = **11**

Q70. Searching Techniques – Practical Efficiency Study

Given the sorted dataset [6, 12, 18, 24, 30, 36, 42, 48], analyze the performance of Linear Search and Binary Search to find the element 36. Clearly interpret the problem and explain the working process of both searching algorithms. Show the step-by-step procedure for each method and count the number of comparisons required. Analyze the time complexity of both approaches. Compare their efficiency for sorted data. Interpret the results and justify which method is preferable for practical use.

Given Sorted Array

[6, 12, 18, 24, 30, 36, 42, 48]

Target Element = **36**

A) Linear Search

Step 1

Compare **6** with **36**

Not Found

Comparisons = **1**

Step 2

Compare **12** with **36**

Not Found

Comparisons = **2**

Step 3

Compare **18** with **36**

Not Found

Comparisons = **3**

Step 4

Compare **24** with **36**

Not Found

Comparisons = **4**

Step 5

Compare **30** with **36**

Not Found

Comparisons = **5**

Step 6

Compare **36** with **36**

Found

Comparisons = **6**

Result

Element **36** found at **index 5 (6th position)**.

Total Comparisons = **6**

B) Binary Search

Initial Values

Low = **0**

High = **7**

Step 1

Mid = (0 + 7) / 2 = **3**

Element = **24**

Since **36 > 24**, search the **right half**.

Low = **4**

High = **7**

Comparisons = **1**

Step 2

Mid = (4 + 7) / 2 = **5**

Element = **36**

Element Found.

Comparisons = **2**

Result

Element **36** found at **index 5 (6th position)**.

Total Comparisons = **2**